159.302 LECTURE 12

Propositional Logic Syntax

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

Propositional Logic

- **What is a logic?**

  A formal language

Propositional Logic

- **What is a logic?**

  A formal language

  - **Syntax** – what expressions are legal

Propositional Logic

- **What is a logic?**

  A formal language

  - **Syntax** – what expressions are legal
  - **Semantics** – what legal expressions mean

Propositional Logic

What is a logic?

A formal language

- **Syntax** – what expressions are legal
- **Semantics** – what legal expressions mean
- **Proof System** – a way of manipulating syntactic expressions to get other syntactic expressions (which will tell us something new)

Propositional Logic

What is a logic?

A formal language

- **Syntax** – what expressions are legal
- **Semantics** – what legal expressions mean
- **Proof System** – a way of manipulating syntactic expressions to get other syntactic expressions (which will tell us something new)

Why proofs?

- Two kinds of inferences an agent might want to make:

Propositional Logic

What is a logic?

A formal language

- **Syntax** – what expressions are legal
- **Semantics** – what legal expressions mean
- **Proof System** – a way of manipulating syntactic expressions to get other sytactic expressions (which will tell us something new)

Why proofs?

- Two kinds of inferences an agent might want to make:
  - **Multiple percepts** => conclusions about the world

Propositional Logic

What is a logic?

A formal language

- **Syntax** – what expressions are legal
- **Semantics** – what legal expressions mean
- **Proof System** – a way of manipulating syntactic expressions to get other sytactic expressions (which will tell us something new)

Why proofs?

- Two kinds of inferences an agent might want to make:
  - **Multiple percepts** => conclusions about the world
  - **Current state & operator** => properties of next state

Propositional Logic

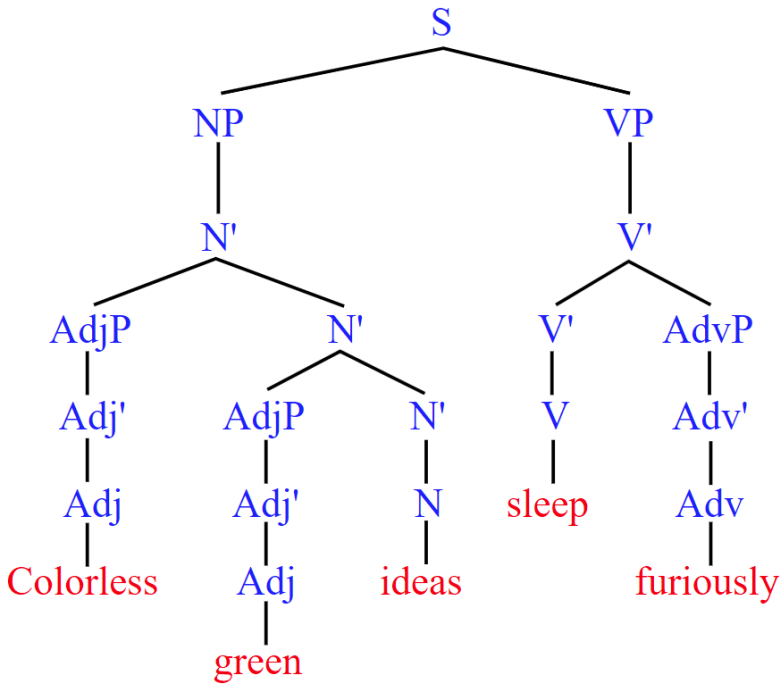
Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.



Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

Sentences (wffs: well-formed formulas)

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

Sentences (wffs: well-formed formulas)

- **true** and **false** are sentences

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

Sentences (wffs: well-formed formulas)

- **true** and **false** are sentences
- Propositional variables are sentences: **P, Q, R, Z**

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

Sentences (wffs: well-formed formulas)

- **true** and **false** are sentences
- Propositional variables are sentences: **P, Q, R, Z**
- If **φ** and **ψ** are sentences, then so are

$$(\phi), \neg\phi, \phi \vee \psi, \phi \wedge \psi, \phi \rightarrow \psi, \phi \leftrightarrow \psi$$

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

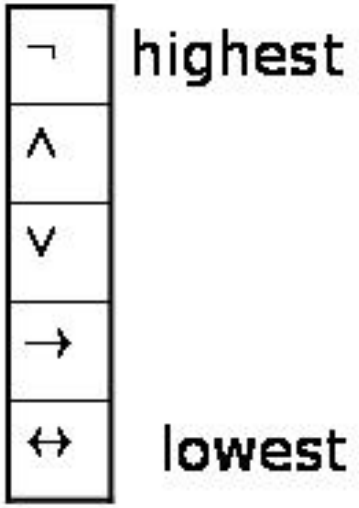
Sentences (wffs: well-formed formulas)

- **true** and **false** are sentences
- Propositional variables are sentences: **P, Q, R, Z**
- If **φ** and **ψ** are sentences, then so are

$$(\phi), \neg\phi, \phi \vee \psi, \phi \wedge \psi, \phi \rightarrow \psi, \phi \leftrightarrow \psi$$

- Nothing else is a sentence

Precedence



$A \vee B \wedge C$	$A \vee (B \wedge C)$
$A \wedge B \rightarrow C \vee D$	$(A \wedge B) \rightarrow (C \vee D)$
$A \rightarrow B \vee C \leftrightarrow D$	$(A \rightarrow (B \vee C)) \leftrightarrow D$

- Precedence rules enable "shorthand" form of sentences, but formally only the fully parenthesized form is legal.
- Syntactically ambiguous forms allowed in shorthand only when semantically equivalent: $A \wedge B \wedge C$ is equivalent to $(A \wedge B) \wedge C$ and $A \wedge (B \wedge C)$

159.302 LECTURE 2

Propositional Logic Semantics

Source: MIT OpenCourseWare

Propositional Logic

Semantics

- Meaning of a sentence is truth value {**t**, **f**}

Propositional Logic

Semantics

- Meaning of a sentence is truth value {**t**, **f**}
- **Interpretation** is an assignment of truth values to the propositional variables

holds(ϕ , i) [Sentence ϕ is **t** in interpretation i]

Propositional Logic

Semantics

- Meaning of a sentence is truth value {**t**, **f**}
- **Interpretation** is an assignment of truth values to the propositional variables

holds(ϕ, i) [Sentence ϕ is **t** in interpretation i]

fails(ϕ, i) [Sentence ϕ is **f** in interpretation i]

Propositional Logic

Semantic Rules

holds(true, i) for all i

Propositional Logic

Semantic Rules

holds(true, i) for all i

fails(false, i) for all i

Propositional Logic

Semantic Rules

holds(true, i) for all i

fails(false, i) for all i

holds($\neg\phi$, i) if and only if fails(ϕ, i) **(negation)**

Propositional Logic

Semantic Rules

holds(true, i) for all i

fails(false, i) for all i

holds($\neg\phi$, i) if and only if fails(ϕ, i) **(negation)**

holds($\phi \wedge \psi$, i) iff holds(ϕ,i) and holds(ψ, i) **(conjunction)**

Propositional Logic

Semantic Rules

holds(true, i) for all i

fails(false, i) for all i

holds($\neg\phi$, i) if and only if fails(ϕ, i) **(negation)**

holds($\phi \wedge \psi$, i) iff holds(ϕ,i) and holds(ψ, i) **(conjunction)**

holds($\phi \vee \psi$, i) iff holds(ϕ,i) or holds(ψ, i) **(disjunction)**

Propositional Logic

Semantic Rules

holds(true, i) for all i

fails(false, i) for all i

holds($\neg\phi$, i) if and only if fails(ϕ, i) **(negation)**

holds($\phi \wedge \psi$, i) iff holds(ϕ,i) and holds(ψ, i) **(conjunction)**

holds($\phi \vee \psi$, i) iff holds(ϕ,i) or holds(ψ, i) **(disjunction)**

holds(P, i) iff i(P) = **t**

fails(P, i) iff i(P) = **f**

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent $\rightarrow$ consequent)**

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent $\rightarrow$ consequent)**

$\phi \leftrightarrow \psi \equiv (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi) \quad (biconditional, equivalence)$

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent → consequent)**

$\phi \leftrightarrow \psi \equiv (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi) \quad (biconditional, equivalence)$

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$Q \rightarrow P$	$P \leftrightarrow Q$
f	f	t	f	f	t	t	t
f	t	t	f	t	t	f	f
t	f	f	f	t	f	t	f
t	t	f	t	t	t	t	t

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent → consequent)**

$\phi \leftrightarrow \psi \equiv (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi) \quad (biconditional, equivalence)$

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$Q \rightarrow P$	$P \leftrightarrow Q$
f	f	t	f	f	t	t	t
f	t	t	f	t	t	f	f
t	f	f	f	t	f	t	f
t	t	f	t	t	t	t	t

Note that implication is not “causality”, if P is **f** then $P \rightarrow Q$ is **t**

Implication doesn't mean "is related" in any kind of real-world way

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent → consequent)**

$\phi \leftrightarrow \psi \equiv (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi) \quad (biconditional, equivalence)$

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$Q \rightarrow P$	$P \leftrightarrow Q$
f	f	t	f	f	t	t	t
f	t	t	f	t	t	f	f
t	f	f	f	t	f	t	f
t	t	f	t	t	t	t	t

Note that implication is not “causality”, if P is **f** then $P \rightarrow Q$ is **t**

Implication doesn't mean "is related" in any kind of real-world way

Propositional Logic

Terminology

A sentence is **valid** iff its truth value is **t** in all interpretations

Valid sentences : $\underline{true}$, $\neg\underline{false}$, $P \vee \neg P$

A sentence is **satisfiable** iff its truth value is **t** in at least one interpretation

Satisfiable sentences: $P, \underline{true}, \neg P$

A sentence is **unsatisfiable** iff its truth value is **f** in all interpretations

Unsatisfiable sentences : $P \wedge \neg P$, $\underline{false}$, $\neg\underline{true}$

All are finitely decidable.

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	
$smoke \rightarrow fire$	satisfiable, not valid	$smoke = t, fire = f$

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	
$smoke \rightarrow fire$	satisfiable, not valid	$smoke = t, fire = f$
$(s \rightarrow f) \rightarrow (\neg s \rightarrow \neg f)$	satisfiable, not valid	$s = f, f = t$ $(s \rightarrow f) = t, (\neg s \rightarrow \neg f) = f$

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	
$smoke \rightarrow fire$	satisfiable, not valid	$smoke = t, fire = f$
$(s \rightarrow f) \rightarrow (\neg s \rightarrow \neg f)$	satisfiable, not valid	$s = f, f = t$ $(s \rightarrow f) = t, (\neg s \rightarrow \neg f) = f$
contrapositive $(s \rightarrow f) \rightarrow (\neg f \rightarrow \neg s)$	valid	

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	
$smoke \rightarrow fire$	satisfiable, not valid	$smoke = t, fire = f$
$(s \rightarrow f) \rightarrow (\neg s \rightarrow \neg f)$	satisfiable, not valid	$s = f, f = t$ $(s \rightarrow f) = t, (\neg s \rightarrow \neg f) = f$
contrapositive $(s \rightarrow f) \rightarrow (\neg f \rightarrow \neg s)$	valid	
$b \vee d \vee (b \rightarrow d)$ $b \vee d \vee \neg b \vee d$	valid	

Satisfiability

- Related to constraint satisfaction

- Given a sentence S, try to find an interpretation i such that ***holds(S,i)***

- Analogous to finding an assignment of values to variables such that the constraints hold

Satisfiability

- Related to constraint satisfaction
- Given a sentence S, try to find an interpretation i such that ***holds(S,i)***
- Analogous to finding an assignment of values to variables such that the constraints hold
- Brute force method: enumerate all interpretations and check

Satisfiability

- Related to constraint satisfaction
- Given a sentence S, try to find an interpretation i such that ***holds(S,i)***
- Analogous to finding an assignment of values to variables such that the constraints hold
- Brute force method: enumerate all interpretations and check
- Better methods:

Heuristic search
Constraint propagation
Stochastic search

Satisfiability Problems

- Scheduling nurses to work in a hospital
  - propositional variables represent for example, that Pat is working on Tuesday at 2pm (= P).
  - constraints on the schedule are represented using logical expressions over the variables
- Finding bugs in software
  - propositional variables represent state of the program
  - use logic to describe how the program works and to assert there is a bug
  - if the sentence is satisfiable, you've found the bug!

159.302 LECTURE 3

Propositional Logic Proof

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

Flowers will be pollinated?

- Imagine we knew that:

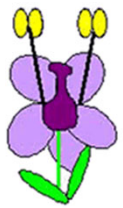
  - If today is Sunny, then the bees will be Foraging $(S \rightarrow F)$
  - If the bees are Foraging, the flowers will be Pollinated $(F \rightarrow P)$
  - Today is sunny (S)

Knowledge Base

Knowledge Base (KB)/Axioms/Premise/Givens/Facts

Should we conclude that the flowers will be pollinated? (P)





Checking Interpretations

S	F	P
t	t	t
t	t	f
t	f	t
t	f	f
f	t	t
f	t	f
f	f	t
f	f	f

8 possible interpretations

start by figuring out what set of interpretations make our original sentences true.

We can see that there are **8 possible interpretations**!

If **P** is true in a set of interpretations that make all the original sentences true, then we can conclude that **P** follows from our original sentences.

Checking Interpretations

interpretations *Knowledge base*

S	F	P	S→F	F→P	S
t	t	t	t	t	t
t	t	f	t	f	t
t	f	t	f	t	t
t	f	f	f	t	t
f	t	t	t	t	f
f	t	f	t	f	f
f	f	t	t	t	f
f	f	f	t	t	f

What set of interpretations make the KB true?

All 3 sentences (Knowledge Base) are true when S=t, F=t, and P=t.

What about the sentence P? Is it ok to conclude that it is also true?

Checking Interpretations

interpretations *sentences* *New sentence*

S	F	P	S→F	F→P	S	P
t	t	t	t	t	t	t
t	t	f	t	f	t	f
t	f	t	f	t	t	t
t	f	f	f	t	t	f
f	t	t	t	t	f	t
f	t	f	t	f	f	f
f	f	t	t	t	f	t
f	f	f	t	t	f	f

P is true in that interpretation!

If **Knowledge Base** is true, in the same set of interpretations that make **P** true (**P** is a new sentence we want to test), then it must be ok to conclude **P** from the KB.



Adding a variable

- Dandelions are blooming (D)

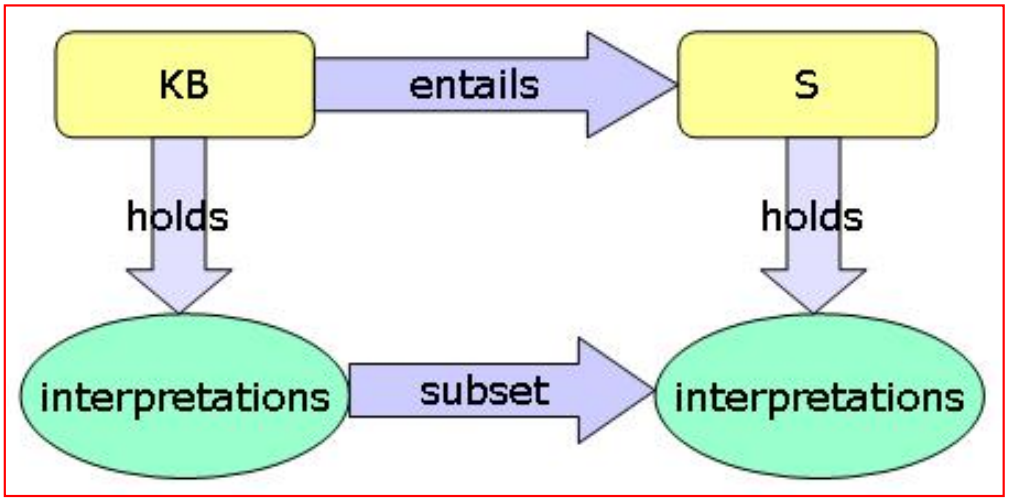
D	S	F	P	S → F	F → P	S	P
t	t	t	t	t	t	t	t
t	t	t	f	t	f	t	f
t	t	f	t	f	t	t	t
t	t	f	f	f	t	t	f
t	f	t	t	t	t	f	t
t	f	t	f	t	f	f	f
t	f	f	t	t	t	f	t
t	f	f	f	t	t	f	f
f	t	t	t	t	t	t	t
f	t	t	f	t	f	t	f
…	…	…	…				

If we add another variable (**D**), we can see that P is still true in two possible interpretations!



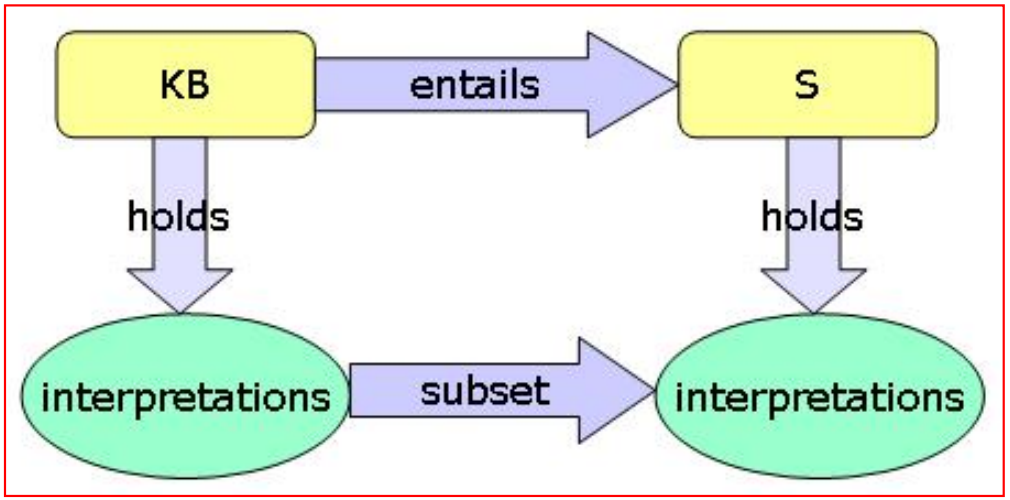
Entailment

•A knowledge base (**KB**) **entails** a sentence **S** iff every interpretation that makes **KB** true also makes **S** true



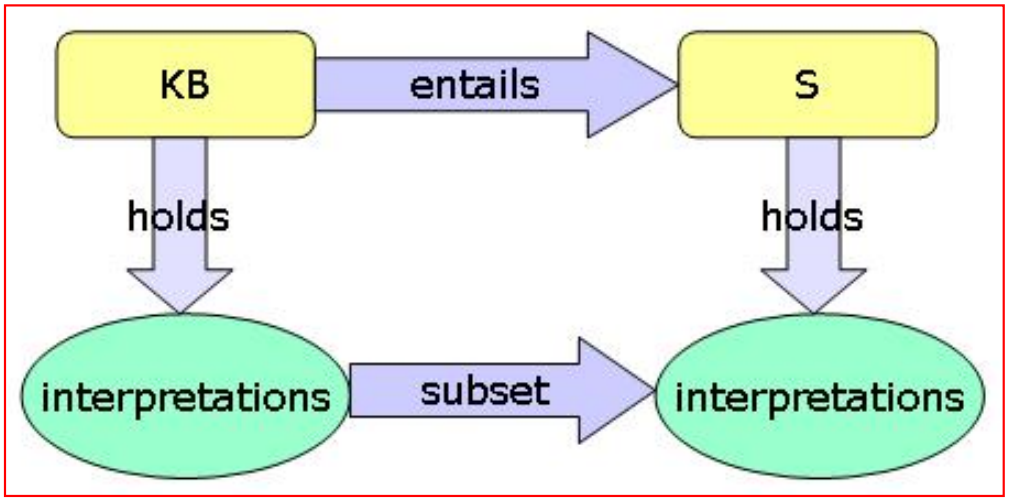
Computing Entailment

- enumerate all interpretations
- select those in which all elements of **KB** are true
- check to see if **S** is true in all of those interpretations



Computing Entailment

- enumerate all interpretations
- select those in which all elements of **KB** are true
- check to see if **S** is true in all of those interpretations

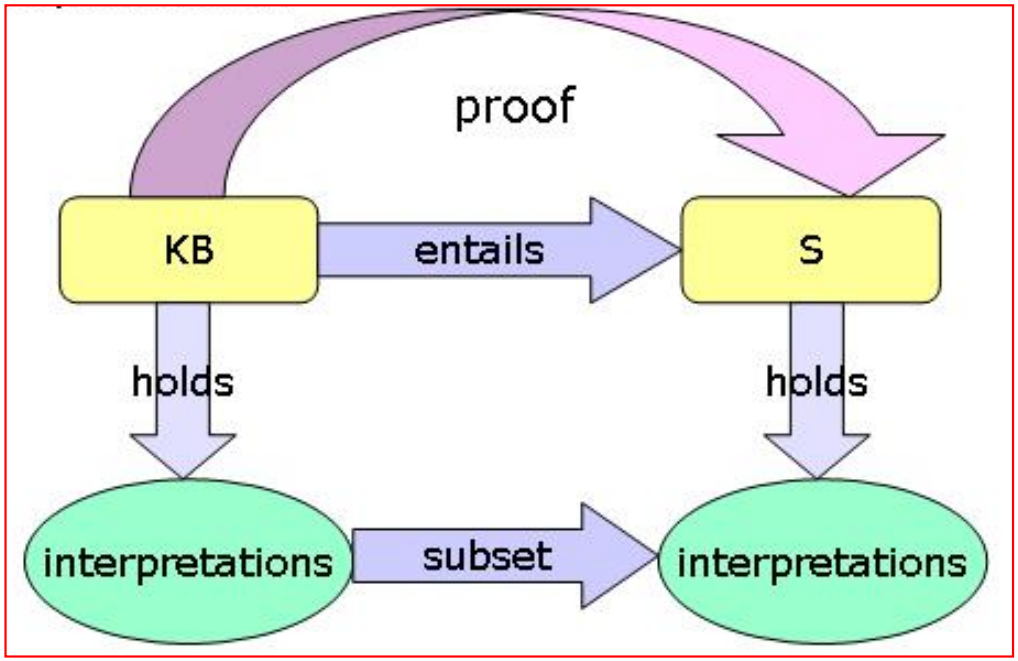


- This is brute force approach
- Way too many interpretations, in general!!
- e.g. (Given 100 propositional variables, there are 2^100 interpretations to check)

1,267,650,600,228,229,401,496,703,205,376 interpretations

Entailment and Proof

• A **proof** is a way to test whether a **KB** entails a sentence, without enumerating all possible interpretations



Proof

- A **proof** is a sequence of sentences
- First ones are premises (**KB**)
- Then, you can write down on the next line the result of applying an inference rule to previous lines
- When **S** is on a line, you have **proved S** from **KB**
- If inference rules are **sound**, then any **S** you can prove from **KB** is entailed by **KB**
- If inference rules are **complete**, then any **S** that is entailed by **KB** can be proved from **KB**

Natural Deduction

- Some **inference rules**:

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Modus ponens

Affirming method

(first written by Aristotle)

Natural Deduction

- Some inference rules:

$$\frac{\alpha \to \beta \quad \alpha}{\beta}$$

Modus ponens

$$\frac{\alpha \to \beta \quad \neg\beta}{\neg\alpha}$$

Modus tolens

Denying method

Natural Deduction

- Some inference rules:

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Modus ponens

$$\frac{\alpha \rightarrow \beta \quad \neg\beta}{\neg\alpha}$$

Modus tolens

$$\frac{\alpha \quad \beta}{\alpha \wedge \beta}$$

And-Introduction

Natural Deduction

- Some inference rules:

$$\frac{\alpha \to \beta \quad \alpha}{\beta}$$

Modus ponens

$$\frac{\alpha \to \beta \quad \neg\beta}{\neg\alpha}$$

Modus tolens

$$\frac{\alpha \quad \beta}{\alpha \wedge \beta}$$

And-Introduction

$$\frac{\alpha \wedge \beta}{\alpha}$$

And-Elimination

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	**1 And-Elim**

$$\frac{\alpha \wedge \beta}{\alpha}$$

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	1 And-Elim
5	R	4,2 Modus-Ponens

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	1 And-Elim
5	R	4,2 Modus-Ponens
6	Q	1 And-Elim

$$\frac{\alpha \wedge \beta}{\alpha}$$

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	1 And-Elim
5	R	4,2 Modus-Ponens
6	Q	1 And-Elim
7	$Q \wedge R$	5,6 And-Intro

$$\frac{\alpha \quad \beta}{\alpha \wedge \beta}$$

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	1 And-Elim
5	R	4,2 Modus-Ponens
6	Q	1 And-Elim
7	$Q \wedge R$	5,6 And-Intro
8	S	7,3 Modus-Ponens

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Proof Systems

• There are many natural deduction systems; they are typically "**proof checkers**", **sound** but not complete

Proof Systems

• There are many natural deduction systems; they are typically "**proof checkers**", **sound** but not complete

• **Natural deduction** uses lots of inference rules which introduces a large branching factor in the search for a proof.

Proof Systems

• There are many natural deduction systems; they are typically "**proof checkers**", **sound** but not complete

• **Natural deduction** uses lots of inference rules which introduces a **large branching factor** in the search for a proof.

• In general, you need to do "**proof by cases**" which introduces even more branching.

Prove R

Step	Formula
1	$P \vee Q$
2	$Q \rightarrow R$
3	$P \rightarrow R$

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

- Single inference rule is a sound and complete proof system

- Requires all sentences to be converted to conjunctive normal form

159.302 LECTURE 4

Propositional Logic
Conjunctive Normal Form (CNF)

Source: MIT OpenCourseWare

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

$A, B, and\ \neg C$ are literals

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

$A, B, and \neg C)$ are literals, each of which is a variable or the negation of a variable.

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

$A, B, and \neg C)$ are literals, each of which is a variable or the negation of a variable.

- Each clause is a requirement that must be satisfied and can be satisfied in multiple ways

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

$A, B, and \neg C)$ are literals, each of which is a variable or the negation of a variable.

- Each clause is a requirement that must be satisfied and can be satisfied in multiple ways

- Every sentence in propositional logic can be written in CNF

Converting to CNF

1. Eliminate arrows using definitions



Converting to CNF

1. Eliminate arrows using definitions
2. Drive in negations using De Morgan's Laws

$$\neg(\phi \vee \varphi) \equiv \neg\phi \wedge \neg\varphi)$$

$$\neg(\phi \wedge \varphi) \equiv \neg\phi \vee \neg\varphi)$$

Converting to CNF

1. Eliminate arrows using definitions

2. Drive in negations using De Morgan's Laws

$$\neg(\phi \vee \varphi) \equiv \neg\phi \wedge \neg\varphi)$$

$$\neg(\phi \wedge \varphi) \equiv \neg\phi \vee \neg\varphi)$$

3. Distribute **or** over **and**

$$(A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

Converting to CNF

1. Eliminate arrows using definitions

2. Drive in negations using De Morgan's Laws

$$\neg(\phi \vee \varphi) \equiv \neg\phi \wedge \neg\varphi)$$

$$\neg(\phi \wedge \varphi) \equiv \neg\phi \vee \neg\varphi)$$

3. Distribute **or** over **and**

$$(A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

4. Every sentence can be converted to CNF, but it may **grow exponentially in size**

CNF Conversion Example

$$(A \vee B) \rightarrow (C \rightarrow D)$$

CNF Conversion Example

$$(A \vee B) \rightarrow (C \rightarrow D)$$

1. Eliminate arrows using definitions

$$\neg(A \vee B) \vee (\neg C \vee D)$$

CNF Conversion Example

$$(A \vee B) \rightarrow (C \rightarrow D)$$

1. Eliminate arrows using definitions

$$\neg(A \vee B) \vee (\neg C \vee D)$$

2. Drive in negations using De Morgan's Laws

$$(\neg A \wedge \neg B) \vee (\neg C \vee D)$$

CNF Conversion Example

$$(A \vee B) \rightarrow (C \rightarrow D)$$

1. Eliminate arrows using definitions

$$\neg(A \vee B) \vee (\neg C \vee D)$$

2. Drive in negations using De Morgan's Laws

$$(\neg A \wedge \neg B) \vee (\neg C \vee D)$$

3. Distribute **or** over **and**

$$(\neg A \vee \neg C \vee D) \wedge (\neg B \vee \neg C \vee D)$$

159.302 LECTURE 5

Propositional Resolution

Source: MIT OpenCourseWare

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Resolution refutation:

- Convert all sentences to CNF, then write each clause down as a premise

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Resolution refutation:

- Convert all sentences to CNF, then write each clause down as a premise
- Negate the desired conclusion (converted to CNF)

This is proof by contradiction!

- Assert the sentence that you're trying to prove is false
- then try to derive a contradiction.

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Resolution refutation:

- Convert all sentences to CNF, then write each clause down as a premise
- Negate the desired conclusion (converted to CNF)
- Apply resolution rule until either
  - Derive false (a contradiction)
  - Can’t apply anymore

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Resolution refutation:

- Convert all sentences to CNF, then write each clause down as a premise
- Negate the desired conclusion (converted to CNF)
- Apply resolution rule until either
  - Derive false (a contradiction)
  - Can't apply anymore

Resolution refutation is sound and complete

- If we derive a contradiction, then the conclusion follows from the axioms
- If we can't apply anymore, then the conclusion cannot be proven from the axioms

Propositional Resolution

Prove R

Step	Formula
1	$P \vee Q$
2	$P \rightarrow R$
3	$Q \rightarrow R$

Step	Formula	Derivation
1	$P \vee Q$	Given
2	$\neg P \vee R$	Given
3	$\neg Q \vee R$	Given
4	$\neg R$	**Negated conclusion**
5	$Q \vee R$	1,2
6	$\neg P$	2,4
7	$\neg Q$	3,4
8	R	5,7
9	▪	4,8

Proof steps

Small box indicates empty clause, meaning false

Propositional Resolution

Prove R

Step	Formula
1	$P \vee Q$
2	$P \rightarrow R$
3	$Q \rightarrow R$

$$\frac{\begin{array}{c} false \vee R \\ \neg R \vee false \end{array}}{false \vee false}$$

Step	Formula	Derivation
1	$P \vee Q$	Given
2	$\neg P \vee R$	Given
3	$\neg Q \vee R$	Given
4	$\neg R$	**Negated conclusion**
5	$Q \vee R$	**1,2**
6	$\neg P$	**2,4**
7	$\neg Q$	**3,4**
8	R	**5,7**
9	▪	**4,8**

Note: disjunction is commutative.

Propositional Resolution

Prove R

Step	Formula
1	$P \vee Q$
2	$P \rightarrow R$
3	$Q \rightarrow R$

Step	Formula	Derivation
1	$P \vee Q$	Given
2	$\neg P \vee R$	Given
3	$\neg Q \vee R$	Given
4	$\neg R$	**Negated conclusion**
5	$Q \vee R$	1,2
6	$\neg P$	2,4
7	$\neg Q$	3,4
8	R	5,7
9	▪	4,8

unnecessary

$$\frac{false \vee R \qquad \neg R \vee false}{false \vee false}$$

The Power of False

Prove Z

Step	Formula
1	P
2	$\neg P$

Step	Formula	Derivation
1	P	Given
2	$\neg P$	Given
3	$\neg Z$	**Negated conclusion**
4	▪	1,2

Note that $(P \wedge \neg P) \rightarrow Z$ is valid

• Any conclusion follows from a contradiction – and so strict logic systems are very brittle.

Example Problem

Prove R

Step	Formula
1	$(P \rightarrow Q) \rightarrow Q$
2	$(P \rightarrow P) \rightarrow R$
3	$(R \rightarrow S) \rightarrow \neg(S \rightarrow Q)$

Convert to CNF

- ¬(¬ P ∨ Q) ∨ Q
- (P ∧ ¬ Q) ∨ Q
- (P ∨ Q) ∧ (¬ Q ∨ Q)
- (P ∨ Q)

- ¬(¬ P ∨ P) ∨ R
- (P ∧ ¬ P) ∨ R
- (P ∨ R) ∧ (¬ P ∨ R)

- ¬(¬ R ∨ S) ∨ ¬ (¬ S ∨ Q)
- (R ∧ ¬ S) ∨ (S ∧ ¬ Q)
- (R ∨ S) ∧ (¬ S ∨ S) ∧ (R ∨ ¬ Q) ∧ (¬ S ∨ ¬ Q)
- (R ∨ S) ∧ (R ∨ ¬ Q) ∧ (¬ S ∨ ¬ Q)

Example Problem

Prove R

Step	Formula
1	$(P \rightarrow Q) \rightarrow Q$
2	$(P \rightarrow P) \rightarrow R$
3	$(R \rightarrow S) \rightarrow \neg(S \rightarrow Q)$

Step		
1	$P \vee Q$	
2	$P \vee R$	
3	$\neg P \vee R$	
4	$R \vee S$	
5	$R \vee \neg Q$	
6	$\neg S \vee \neg Q$	
7	$\neg R$	**Neg**

Example Problem

Prove R

Step	Formula
1	$(P \to Q) \to Q$
2	$(P \to P) \to R$
3	$(R \to S) \to \neg(S \to Q)$

Step		
1	$P \vee Q$	
2	$P \vee R$	
3	$\neg P \vee R$	
4	$R \vee S$	
5	$R \vee \neg Q$	
6	$\neg S \vee \neg Q$	
7	$\neg R$	Neg
8	S	4,7
9	$\neg Q$	6,8
10	P	1,9
11	R	3,10
12	■	7,11

Proof Strategies

Unit Preference: prefer a resolution step involving a unit clause (clause with one literal).

- Produces a shorter clause – which is good since we are trying to produce a zero-length clause, that is, a contradiction.

Set of support: choose a resolution involving the negated goal or any clause derived from the negated goal.

- We're trying to produce a contradiction that follows from the negated goal, so these are "relevant" clauses
- If a contradiction exists, one can find one using the set-of-support strategy.

References

MIT Opencourseware

Logic for Computer Science and Artificial Intelligence

- http://onlinelibrary.wiley.com/book/10.1002/9781118604182